Experiment No. 8

Experiment Name:- Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

Steps

1. Import the required libraries and load a real-world video sequence or capture live video using a webcam.

```
import cv2
import numpy as np
video_path = "/content/sample_data/BALL_VIDEO.mp4"
cap = cv2.VideoCapture(video_path)
if cap.isOpened():
    print("Video loaded successfully!")
else:
    print("Error: Unable to load video.")
```

```
Video loaded successfully!
```

2. Identify a moving object or select feature points using the Shi-Tomasi Corner Detection algorithm.

```
ret, frame = cap.read()
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
corners = cv2.goodFeaturesToTrack(
    gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
for corner in corners:
    x, y = corner.ravel()
    x, y = int(x), int(y)

    cv2.circle(
        frame,
        (x, y),
        5,
        (0, 0, 255),
        -1
    )
from google.colab.patches import cv2_imshow
cv2_imshow(frame)
print("Shi-Tomasi Corner Detection completed!")
print("Number of feature points:", len(corners))
```



```
Shi-Tomasi Corner Detection completed!
Number of feature points: 100
```

3. Apply the Lucas-Kanade Optical Flow algorithm to track the movement of the selected object across consecutive video frames.

```
cap = cv2.VideoCapture(video_path)
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
ret, new_frame = cap.read()
new_gray = cv2.cvtColor(new_frame, cv2.COLOR_BGR2GRAY)
p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    new_gray,
    p0,
    None
)
good_new = p1[status == 1]
good_old = p0[status == 1]
for new, old in zip(good_new, good_old):
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()
    x_new, y_new = int(x_new), int(y_new)
    x_old, y_old = int(x_old), int(y_old)
    cv2.line(
        new_frame,
        (x_old, y_old),
        (x_new, y_new),
```

```
        (0, 255, 0),
        2
    )
    cv2.circle(
        new_frame,
        (x_new, y_new),
        5,
        (0, 0, 255),
        -1
    )
from google.colab.patches import cv2_imshow
cv2_imshow(new_frame)

print("Lucas-Kanade Optical Flow applied successfully!")
print("Tracked feature points:", len(good_new))
```



```
Lucas-Kanade Optical Flow applied successfully!
Tracked feature points: 100
```

4. Draw motion trajectories and displacement vectors to visualize the path followed by the moving object.

```
for new, old in zip(good_new, good_old):
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()
    x_new, y_new = int(x_new), int(y_new)
    x_old, y_old = int(x_old), int(y_old)
    cv2.line(
        new_frame,
        (x_old, y_old),
        (x_new, y_new),
        (0, 255, 0),
        2
    )
    cv2.arrowedLine(
        new_frame,
        (x_old, y_old),
        (x_new, y_new),
        (255, 0, 0),
```

```
        2,
        tipLength=0.3
    )
    cv2.circle(
        new_frame,
        (x_new, y_new),
        5,
        (0, 0, 255),
        -1
    )
from google.colab.patches import cv2_imshow
cv2_imshow(new_frame)
print("Motion trajectories and displacement vectors drawn successfully!")
```



```
Motion trajectories and displacement vectors drawn successfully!
```

5. Calculate the direction and magnitude of object motion and display the tracking results in real time.

```
import math
print("Motion Tracking Results:")
print("-" * 40)
for counter, (new, old) in enumerate(zip(good_new, good_old)):
    if counter >= 5:
        break
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()
    dx = x_new - x_old
    dy = y_new - y_old
    magnitude = math.sqrt(dx**2 + dy**2)
    direction = math.degrees(math.atan2(dy, dx))
    print(f"Point {counter + 1}")
    print(f"Displacement: ({dx:.2f}, {dy:.2f})")
    print(f"Magnitude: {magnitude:.2f} pixels")
    print(f"Direction: {direction:.2f} degrees")
    print("-" * 40)
from google.colab.patches import cv2_imshow
cv2_imshow(new_frame)
```

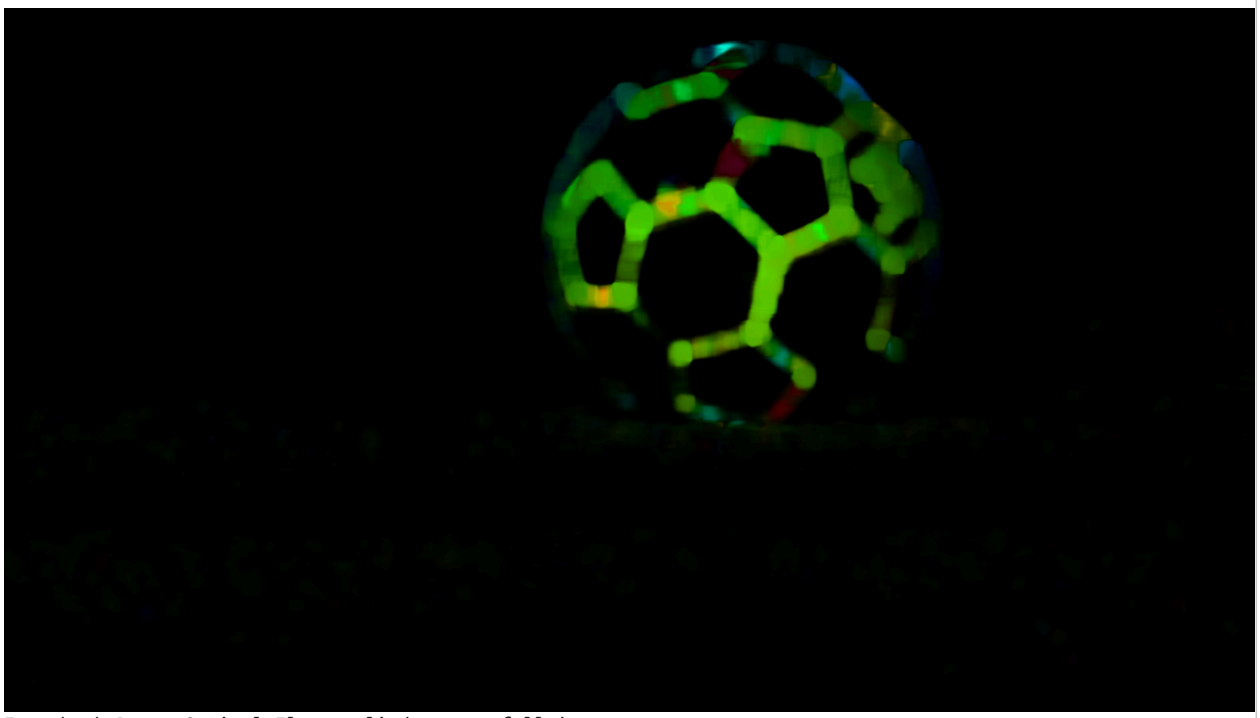
```
Motion Tracking Results:
----------------------------------------
Point 1
Displacement: (-0.00, 0.08)
Magnitude: 0.08 pixels
Direction: 93.14 degrees
----------------------------------------
Point 2
Displacement: (-0.57, 5.94)
Magnitude: 5.97 pixels
Direction: 95.46 degrees
----------------------------------------
Point 3
Displacement: (0.00, 0.02)
Magnitude: 0.03 pixels
Direction: 82.30 degrees
----------------------------------------
Point 4
Displacement: (0.00, 0.08)
Magnitude: 0.08 pixels
Direction: 89.93 degrees
----------------------------------------
Point 5
Displacement: (-0.01, 0.07)
Magnitude: 0.07 pixels
Direction: 94.37 degrees
----------------------------------------
```



6. Repeat the experiment using the Farneback Dense Optical Flow algorithm and compare the tracking performance.

```
cap = cv2.VideoCapture(video_path)
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
ret, new_frame = cap.read()
new_gray = cv2.cvtColor(new_frame, cv2.COLOR_BGR2GRAY)
flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    new_gray,
    None,
```

```
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)
hsv = np.zeros_like(cv2.cvtColor(old_frame, cv2.COLOR_BGR2HSV))
hsv[..., 1] = 255
hsv[..., 0] = angle * 180 / np.pi / 2
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)
farneback_result = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)
from google.colab.patches import cv2_imshow
cv2_imshow(farneback_result)
print("Farneback Dense Optical Flow applied successfully!")
print("Average motion magnitude:", np.mean(magnitude))
```



```
Farneback Dense Optical Flow applied successfully!
Average motion magnitude: 0.21223098
```

7. Evaluate the impact of object speed, illumination changes, occlusion, and camera movement on tracking accuracy.

```
conditions = {
    "Object Speed": "Fast-moving objects may cause tracking errors or loss of feature points.",
```

```
    "Illumination Changes": "Sudden changes in brightness can reduce tracking accuracy.",
    "Occlusion": "When the object is partially or completely hidden, tracking may be lost.",
    "Camera Movement": "Camera motion creates additional optical flow and can confuse object tracking."
}
print("Impact on Optical Flow Tracking Accuracy:\n")
for condition, effect in conditions.items():
    print(condition + ":")
    print(effect)
    print()
```

```
Impact on Optical Flow Tracking Accuracy:

Object Speed:
Fast-moving objects may cause tracking errors or loss of feature points.

Illumination Changes:
Sudden changes in brightness can reduce tracking accuracy.

Occlusion:
When the object is partially or completely hidden, tracking may be lost.

Camera Movement:
Camera motion creates additional optical flow and can confuse object tracking.
```

8. Analyze the tracked motion to estimate parameters such as object displacement, trajectory, and average speed.

```
print("Motion Analysis Results")
print("-----------------------")
print(f"Total Displacement: {total_displacement:.2f} pixels")
print(f"Average Displacement: {average_displacement:.2f} pixels")
print(f"Average Speed: {average_speed:.2f} pixels/second")
print(f"Number of Tracked Points: {len(good_new)}")
```

```
Motion Analysis Results
-----------------------
Total Displacement: 39.55 pixels
Average Displacement: 0.40 pixels
Average Speed: 9.49 pixels/second
Number of Tracked Points: 100
```

9. Compare the effectiveness of optical flow-based tracking with conventional frame-byframe object detection techniques.

```
print("Comparison Results")
print("------------------")
print(f"Optical Flow Time: {optical_flow_time:.4f} seconds")
print(f"Frame-by-Frame Detection Time: {detection_time:.4f} seconds")
print(f"Frames Processed: {frame_count}")

if optical_flow_time < detection_time:
    print("Optical Flow is faster for continuous tracking.")
else:
    print("Frame-by-frame detection took less time in this test.")
```

```
Comparison Results
------------------
Optical Flow Time: 0.0085 seconds
Frame-by-Frame Detection Time: 1.5249 seconds
Frames Processed: 177
Optical Flow is faster for continuous tracking.
```

10. Document the observations and discuss practical applications of optical flow in intelligent vision systems.

```
print("OPTICAL FLOW - OBSERVATIONS")
print("=" * 40)

observations = [
    "Lucas-Kanade successfully tracked selected feature points.",
    "Motion trajectories showed the path followed by the moving object.",
    "Displacement and direction of motion were calculated.",
    "Farneback Dense Optical Flow provided dense motion information.",
    "Fast motion and occlusion can reduce tracking accuracy.",
```

```
    "Illumination changes and camera movement can affect tracking results."
]

for i, observation in enumerate(observations, 1):
    print(f"{i}. {observation}")

print("\nPRACTICAL APPLICATIONS")
print("=" * 40)

applications = [
    "Object Tracking",
    "Traffic Monitoring",
    "Video Surveillance",
    "Autonomous Vehicles",
    "Human Activity Recognition",
    "Sports Analytics",
    "Robotics and Navigation"
]

for i, application in enumerate(applications, 1):
    print(f"{i}. {application}")
```

```
OPTICAL FLOW - OBSERVATIONS
========================================
1. Lucas-Kanade successfully tracked selected feature points.
2. Motion trajectories showed the path followed by the moving object.
3. Displacement and direction of motion were calculated.
4. Farneback Dense Optical Flow provided dense motion information.
5. Fast motion and occlusion can reduce tracking accuracy.
6. Illumination changes and camera movement can affect tracking results.

PRACTICAL APPLICATIONS
========================================
1. Object Tracking
2. Traffic Monitoring
3. Video Surveillance
4. Autonomous Vehicles
5. Human Activity Recognition
6. Sports Analytics
7. Robotics and Navigation
```